



# JWT



**{desafío}**  
latam\_



**Activen las cámaras los que puedan y  
pasemos asistencia**

## ***Implementar la autenticación y autorización con JWT en una aplicación de React.***

**{desafío}**  
**latam\_**

- Unidad 1: Introducción a React.
- Unidad 2: Estados de los componentes y eventos.
- Unidad 3: Renderización dinámica de componentes.
- Unidad 4: Consumo de APIs con React.
- Unidad 5: React Router I
- Unidad 6: Context
- Unidad 7: React Router II
- Unidad 8: JWT



Te encuentras aquí



# Inicio

{desafío}  
latam\_



*/\* Identifica qué es la autenticación y autorización de usuarios. \*/*

*/\* Reconoce el uso de JWT como estándar para transmitir información segura. \*/*

*/\* Reconoce el funcionamiento de JWT. \*/*

*/\* Implementa JWT en una aplicación React. \*/*

# Objetivos

# ¿Qué es JWT?

- JWT (JSON Web Token) es un estándar abierto basado en JSON que define una forma compacta y autónoma para transmitir información de forma segura entre dos partes como un objeto JSON.
- Esta información puede ser verificada y confiable porque está firmada digitalmente.
- JWT se utiliza para autenticar usuarios y autorizarlos a acceder a recursos específicos.



# Composición de un token JWT

*Un token JWT consta de tres partes separadas por un punto.*

- **Encabezado (Header):** Contiene el tipo de token y el algoritmo de firma.
- **Cuerpo (Payload):** Contiene la información del usuario y los datos adicionales.
- **Firma (Signature):** Contiene la firma digital que se utiliza para verificar la autenticidad del token.

## Ejemplo de token generado

*Un token JWT consta de tres partes separadas por un punto.*

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c
```



# ¿Cómo funciona JWT?

*Un token JWT consta de tres partes separadas por un punto.*

1. El cliente envía un nombre de usuario y contraseña al servidor.
2. El servidor verifica las credenciales y genera un token JWT.
3. El servidor envía el token JWT al cliente.
4. El cliente almacena el token JWT y lo envía en el encabezado de la solicitud HTTP.
5. El servidor verifica el token JWT y autoriza al cliente a acceder a los recursos.

*/\** Identifica qué es la autenticación y autorización de usuarios. *\*/* ✓

*/\** Reconoce el uso de JWT como estándar para transmitir información segura. *\*/* ✓

*/\** Reconoce el funcionamiento de JWT. *\*/* ✓

*/\** Implementa JWT en una aplicación React. *\*/*

# Objetivos

## Ejercicio

Repite los pasos:

1. Descarga el “Material de apoyo - Implementación de JWT en una aplicación de React”.
2. Instala las dependencias con: `npm install` y ejecuta el modo desarrollo con `npm run dev`.
3. Vamos a generar el siguiente componente Login que se encargará de hacer login en nuestra aplicación. Antes de comenzar, verifica que tienes levantado el servidor (backend) que estás utilizando en el desarrollo de tus hitos.

El endpoint `/api/auth/login` recibe un email y una contraseña y devuelve un token JWT.

## Ejercicio

### ¡Manos al teclado!



## Ejercicio

Repite los pasos:

4. Para probar, utiliza las siguientes credenciales:

```
{  
  "email": "test@test.com",  
  "password": "123123"  
}
```

## Ejercicio

### ¡Manos al teclado!



- Importamos las dependencias necesarias y definimos el componente Login.
- Creamos dos estados email y password para almacenar los valores de los campos de entrada.
- Creamos una función handleSubmit que se ejecuta cuando se envía el formulario.
- Hacemos una petición POST a la ruta /api/auth/login con el email y la contraseña.
- Almacenamos el token JWT en el localStorage.
- Creamos un formulario con dos campos de entrada para el email y la contraseña.
- Al hacer clic en el botón Login, se ejecuta la función handleSubmit.

En este ejemplo estamos persistiendo el token JWT en el localStorage, así podremos acceder a él en cualquier parte de la aplicación.

## Ejercicio

Repite los pasos:

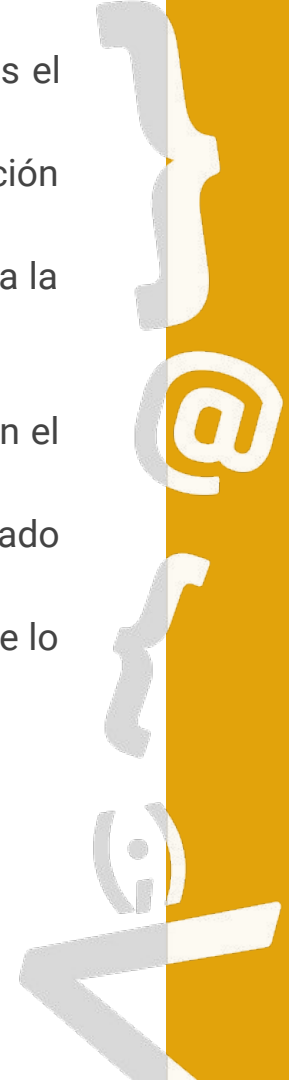
5. Una vez que hemos obtenido el token JWT, podemos utilizarlo para acceder a rutas protegidas en nuestra aplicación. Vamos a crear un componente Profile que se encargará de mostrar el perfil del usuario autenticado. `src\pages\Profile.jsx`

## Ejercicio

### ¡Manos al teclado!



- Importamos las dependencias necesarias y definimos el componente Profile.
- Creamos un estado user para almacenar la información del usuario.
- Utilizamos el hook useEffect para hacer una petición a la ruta /api/auth/me al cargar el componente.
- Obtenemos el token JWT del localStorage.
- Hacemos una petición GET a la ruta /api/auth/me con el token JWT en el encabezado.
- Almacenamos la información del usuario en el estado user.
- Mostramos el email del usuario si está autenticado, de lo contrario mostramos un mensaje.

A vertical decorative bar on the right side of the slide, featuring a yellow background and a white border. It contains stylized white icons of a closing curly brace '}', an '@' symbol, an opening curly brace '{', and a semi-transparent '@' symbol.

Entendamos el código

**/\* Custom hooks \*/**



# ¿Qué son los custom hooks?

Los Custom Hooks son una característica de React que nos permite extraer lógica de componentes en funciones reutilizables.

En el proyecto de apoyo se encuentra un Custom Hook llamado ``useInput`` que nos permite manejar el estado de un campo de entrada en un formulario, además de un ejemplo de cómo utilizarlo en ``RegisterWithCustomHooks.jsx``.

## Ejercicio

Repite los pasos:

6. Un ejemplo de Custom Hook sería el siguiente: `src\hooks\useInput.jsx`.

- En este ejemplo, hemos creado un Custom Hook llamado `useInput` que nos permite manejar el estado de un campo de entrada en un formulario.
- El Custom Hook `useInput` recibe un valor inicial y devuelve un objeto con dos propiedades: `value` y `onChange`.
- `value` es el valor actual del campo de entrada y `onChange` es una función que se ejecuta cuando el campo de entrada cambia.

## Ejercicio

### ¡Manos al teclado!



## Ejercicio

Repite los pasos:

7. Para utilizar este Custom Hook en un componente, simplemente importamos la función y la llamamos:

De esta manera, podemos reutilizar la lógica de manejo de estado en múltiples componentes sin tener que repetir el código. En este caso, llamamos a `useInput` dos veces para manejar el estado de dos campos de entrada en un formulario. Luego utilizamos las propiedades `value` y `onChange` en los campos de entrada.

## Ejercicio

### ¡Manos al teclado!



# Análisis del destructuring realizado

- `{...email}` es una forma abreviada de escribir `value={email.value} onChange={email.onChange}`.
- `{...password}` es una forma abreviada de escribir `value={password.value} onChange={password.onChange}`.
- Esto se conoce como "destructuring" en JavaScript y nos permite pasar múltiples propiedades a un componente de forma más concisa.
- Los Custom Hooks son una herramienta poderosa que nos permite escribir código más limpio y modular en nuestras aplicaciones de React.

Ideas importantes:

- Hemos aprendido cómo implementar la autenticación y autorización con JWT en una aplicación de React. Hemos visto cómo hacer login y obtener el perfil del usuario autenticado utilizando tokens JWT.
- Además hemos conocido los Custom Hooks en React, una característica que nos permite extraer lógica de componentes en funciones reutilizables.

A vertical line separates the white text area from the blue background. It features a series of white and light blue symbols: a closing curly brace '}', an email symbol '@', an opening curly brace '{', and a person icon, arranged vertically.

## Reflexionemos



Cierre

{desafío}  
latam\_



¿Existe algún concepto que no  
hayas comprendido?

Reflexionemos

¿Tienen alguna duda respecto al Hito?





*Academia de  
talentos digitales*

[www.desafiolatam.com](http://www.desafiolatam.com)



/DesafioLatam



/DesafioLatam



/DesafioLatam



/DesafioLatam